

LAPORAN PRAKTIKUM

STRUKTUR DATA

“BFS DAN DFS”



disusun Oleh:

NAIRA RAMADHANI HALIL

2511533027

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T.

Asisten Praktikum:

ZAHRAN AZARIA ANVAYA

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena berkat rahmat dan karunia-Nya, laporan praktikum dengan judul “BFS dan DFS” dapat diselesaikan tepat waktu.

Laporan ini disusun untuk memenuhi salah satu tugas mata kuliah Praktikum Struktur Data, sekaligus sebagai sarana pembelajaran dalam memahami konsep dasar pemrograman khususnya materi BFS dan DFS pada Bahasa Java.

Pada kesempatan ini, penulis menyampaikan terimakasih kepada:

1. Bapak Dr. Wahyudi, S.T., M.T. selaku dosen pengampu mata kuliah Praktikum Struktur Data yang telah memberikan bimbingan dan arahan.
2. Kak Zahran Azaria Anvaya selaku asisten praktikum kelas A yang telah membantu pelaksanaan praktikum.
3. Teman-teman mahasiswa yang telah membantu dan memberi dukungan serta berdiskusi bersama dalam menyelesaikan laporan ini.

Penulis menyadari bahwa laporan ini masih jauh dari kata sempurna. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan demi penyempurnaan laporan di masa mendatang.

Akhir kata, semoga laporan ini dapat memberikan manfaat bagi pembaca dan menambah wawasan mengenai pemrograman serta struktur data pada Java.

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan Praktikum.....	1
1.3 Manfaat Praktikum.....	2
BAB II PEMBAHASAN.....	3
2.1 Langkah Kerja Praktikum	3
2.2 Analisis Hasil dan Pembahasan	14
BAB III PENUTUP.....	17
3.1 Kesimpulan	17
DAFTAR PUSTAKA	

BAB I

PENDAHULUAN

1.1 Latar Belakang

Struktur data merupakan salah satu materi penting dalam pemrograman yang digunakan untuk mengatur dan mengelola data agar dapat diproses secara efisien. Salah satu bentuk struktur data yang sering digunakan adalah **tree** dan **graph**. Pada graph, data direpresentasikan dalam bentuk simpul (vertex) dan sisi (edge) yang saling terhubung. Struktur graph banyak digunakan dalam kehidupan sehari-hari, seperti pada sistem navigasi, jaringan komputer, media sosial, dan pencarian rute terpendek.

Untuk melakukan penelusuran atau pencarian pada graph, terdapat beberapa algoritma yang umum digunakan, yaitu **Breadth First Search (BFS)** dan **Depth First Search (DFS)**. Algoritma BFS bekerja dengan menelusuri simpul berdasarkan tingkat kedekatan dari simpul awal menggunakan struktur data queue, sedangkan DFS melakukan penelusuran hingga ke simpul terdalam terlebih dahulu sebelum kembali ke simpul sebelumnya dengan memanfaatkan konsep rekursi atau stack.

Pada praktikum ini dilakukan implementasi struktur data tree dan graph menggunakan bahasa pemrograman Java. Selain itu, dilakukan juga penerapan algoritma BFS dan DFS untuk memahami cara kerja penelusuran simpul pada graph. Dengan memahami kedua algoritma tersebut, mahasiswa dapat mengetahui perbedaan proses penelusuran serta penerapannya dalam menyelesaikan berbagai permasalahan yang berkaitan dengan struktur data graph.

1.2 Tujuan Praktikum

1. Memahami konsep dasar struktur data tree dan graph dalam pemrograman.
2. Mempelajari cara membangun dan merepresentasikan graph menggunakan bahasa pemrograman Java.
3. Memahami prinsip kerja algoritma Breadth First Search (BFS) dan Depth First Search (DFS).
4. Mengimplementasikan algoritma BFS dan DFS untuk melakukan penelusuran pada graph.
5. Menganalisis urutan penelusuran simpul yang dihasilkan oleh algoritma BFS dan DFS.

6. Melatih kemampuan dalam mengembangkan program yang berkaitan dengan struktur data dan algoritma penelusuran.

1.3 Manfaat Praktikum

1. Mahasiswa dapat memahami konsep dan implementasi struktur data tree serta graph secara lebih mendalam.
2. Mahasiswa mampu menerapkan algoritma BFS dan DFS dalam menyelesaikan masalah yang melibatkan proses pencarian dan penelusuran data.
3. Mahasiswa memperoleh pengalaman dalam mengimplementasikan struktur data dan algoritma menggunakan bahasa pemrograman Java.
4. Mahasiswa dapat membedakan karakteristik dan cara kerja BFS serta DFS dalam proses penelusuran graph.
5. Mahasiswa memiliki dasar yang lebih kuat untuk mempelajari materi lanjutan yang berkaitan dengan graph, seperti pencarian jalur terpendek, spanning tree, dan analisis jaringan.
6. Mahasiswa dapat meningkatkan kemampuan berpikir logis dan analitis dalam menyelesaikan permasalahan komputasi.

BAB II PEMBAHASAN

2.1 Langkah Kerja Praktikum

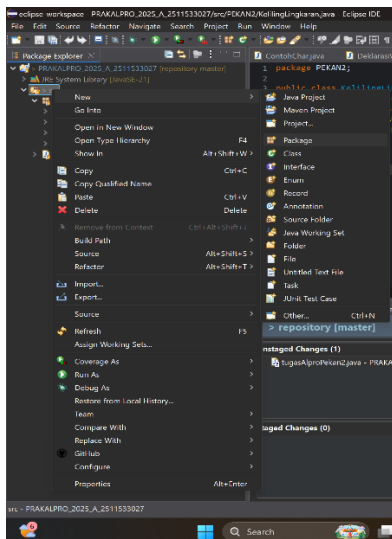
1. Persiapan Awal

- Buka apl Eclipse IDE di laptop atau di komputer.

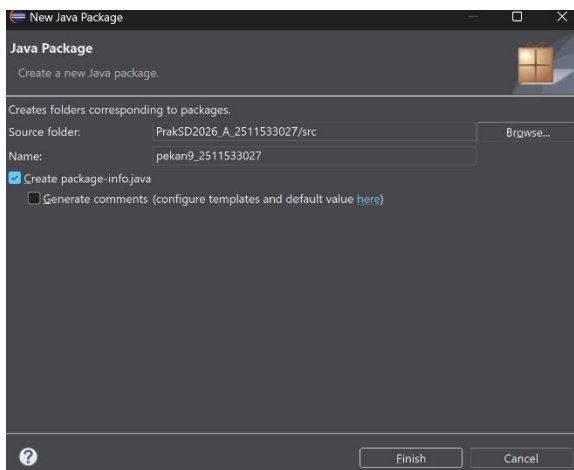


2. Membuat Package

- Klik kanan pada folder **src**, lalu pilih **New**, terakhir klik **Package**.

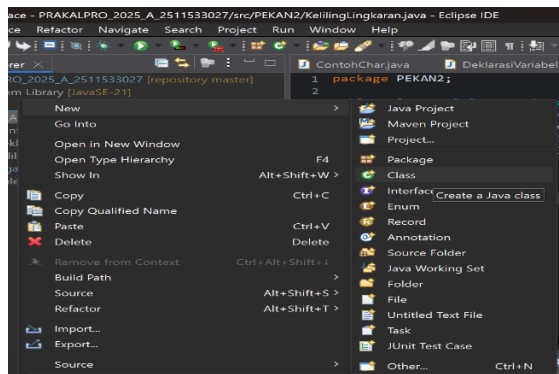


- Beri nama package **pekan9_NIM**.



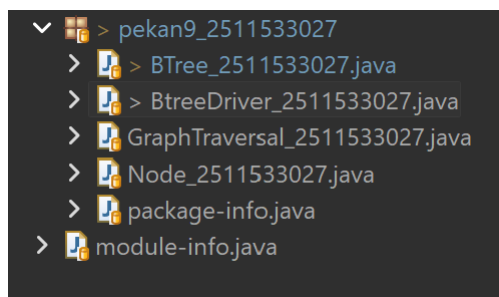
3. Membuat Class Baru untuk Setiap Percobaan

- a) Klik kanan pada package **pekan9_NIM**, lalu pilih **New**, terakhir klik **Class**.



- b) Buat beberapa kelas sesuai percobaan.

- BTree_2511533027.java
- BtreeDriver_2511533027.java
- GraphTraversal_2511533027.java
- Node_2511533027.java



4. Menulis Kode Program

1. BTree_2511533027.java

- Program ini digunakan untuk mengimplementasikan struktur data Binary Tree menggunakan bahasa pemrograman Java.
- Program berfungsi untuk mengelola operasi dasar pada Binary Tree, seperti pencarian data, menghitung jumlah simpul, dan melakukan traversal pohon.
- Method `search_3027()` digunakan untuk mencari data tertentu pada setiap simpul pohon secara rekursif.
- Method `countNodes_3027()` digunakan untuk menghitung jumlah seluruh simpul yang terdapat pada Binary Tree.
- Method `printInorder_3027()`, `printPreOrder_3027()`, dan `printPostOrder_3027()` digunakan untuk menampilkan isi pohon berdasarkan metode traversal yang berbeda.

- Program juga menyediakan method untuk mengatur root dan simpul aktif melalui setRoot_3027() dan setCurrent_3027().
- Hasil akhir program berupa pengelolaan dan penelusuran data pada Binary Tree yang dapat ditampilkan dalam berbagai bentuk traversal.

```
package pekan9_2511533027;

public class BTree_2511533027 {
    private Node_2511533027 root_3027;
    private Node_2511533027 currentNode_3027;
    public BTree_2511533027() {
        root_3027 = null;
    }

    public boolean search(int data_3027) {
        return search_3027(root_3027, data_3027);
    }

    private boolean search_3027(Node_2511533027 node_3027, int data_3027) {
        if (node_3027.getData_3027() == data_3027)
            return true;
        if (node_3027.getLeft_3027() != null)
            if (search_3027(node_3027.getLeft_3027(), data_3027))
                return true;
        if (node_3027.getRight_3027() != null)
            if (search_3027(node_3027.getRight_3027(), data_3027))
                return true;
        return false;
    }

    public void printInorder_3027() {
        root_3027.printInorder_3027(root_3027);
    }

    public void printPreorder_3027() {
        root_3027.printPreorder_3027(root_3027);
    }
}
```



```

public void printPostorder_3027() {
    root_3027.printPostorder_3027(root_3027);
}

public Node_2511533027 getRoot_3027() {
    return root_3027;
}

public boolean isEmpty_3027() {
    return root_3027 == null;
}

public int countNodes_3027() {
    return countNodes_3027(root_3027);
}

private int countNodes_3027(Node_2511533027 node_3027) {
    int count_3027 = 1;
    if (node_3027 == null) {
        return 0;
    } else {
        count_3027
countNodes_3027(node_3027.getLeft_3027());
        count_3027
countNodes_3027(node_3027.getRight_3027());
        return count_3027;
    }
}

public void print_3027() {
    root_3027.print_3027();
}

public Node_2511533027 getCurrent_3027() {
    return currentNode_3027;
}

public void setCurrent_3027(Node_2511533027 node_3027) {
    this.currentNode_3027 = node_3027;
}

```

```

    public void setRoot_3027(Node_2511533027 root_3027) {
        this.root_3027 = root_3027;
    }
}

```

2. BtreeDriver_2511533027.java

- Program ini digunakan untuk menguji implementasi Binary Tree yang telah dibuat pada kelas BTree dan Node.
- Program bekerja dengan membuat beberapa simpul dan menghubungkannya sehingga membentuk sebuah struktur Binary Tree.
- Method main() digunakan sebagai titik awal eksekusi program dan berisi proses pembuatan serta pengaturan hubungan antar simpul.
- Program menetapkan simpul bernilai 1 sebagai root dan menambahkan beberapa simpul lain sebagai anak kiri maupun anak kanan.
- Setelah pohon terbentuk, program menghitung jumlah simpul yang ada pada Binary Tree menggunakan method countNodes_3027().
- Program menampilkan hasil traversal Inorder, Preorder, dan Postorder untuk menunjukkan urutan kunjungan simpul berdasarkan metode traversal yang berbeda.
- Hasil akhir program berupa tampilan jumlah simpul, urutan traversal, dan representasi struktur Binary Tree dalam bentuk pohon.

```

package pekan9_2511533027;

public class BtreeDriver_2511533027 {
    public static void main(String[] args) {
        // Membuat Pohon
        BTree_2511533027 tree_3027 = new BTree_2511533027();
        System.out.print("Jumlah simpul awal pohon: ");
        System.out.println(tree_3027.countNodes_3027());

        // Menambahkan simpul data 1
        Node_2511533027 root_3027 = new Node_2511533027(1);

        // Menjadikan simpul 1 sebagai root
        tree_3027.setRoot_3027(root_3027);
    }
}

```

```

        System.out.println("Jumlah simpul jika hanya ada root");
        System.out.println(tree_3027.countNodes_3027());
        Node_2511533027 node2_3027 = new Node_2511533027(2);
        Node_2511533027 node3_3027 = new Node_2511533027(3);
        Node_2511533027 node4_3027 = new Node_2511533027(4);
        Node_2511533027 node5_3027 = new Node_2511533027(5);
        Node_2511533027 node6_3027 = new Node_2511533027(6);
        Node_2511533027 node7_3027 = new Node_2511533027(7);
        Node_2511533027 node8_3027 = new Node_2511533027(8);
        Node_2511533027 node9_3027 = new Node_2511533027(9);
        root_3027.setLeft_3027(node2_3027);
        node2_3027.setLeft_3027(node4_3027);
        node2_3027.setRight_3027(node5_3027);
        node4_3027.setRight_3027(node8_3027);
        root_3027.setRight_3027(node3_3027);
        node3_3027.setLeft_3027(node6_3027);
        node3_3027.setRight_3027(node7_3027);
        node6_3027.setLeft_3027(node9_3027);

        // Set root
        tree_3027.setCurrent_3027(tree_3027.getRoot_3027());
        System.out.println("menampilkan simpul terakhir: ");

        System.out.println(tree_3027.getCurrent_3027().getData_3027());
        System.out.println("Jumlah simpul; setelah simpul 7
ditambahkan");
        System.out.println(tree_3027.countNodes_3027());
        System.out.println("InOrder: ");
        tree_3027.printInorder_3027();
        System.out.println("\nPreorder: ");
        tree_3027.printPostorder_3027();
        System.out.println("\nMenampilkan simpul dalam bentuk pohon");
        tree_3027.print_3027();
    }
}

```

3. GraphTraversal_2511533027.java

- Program ini digunakan untuk memahami proses penelusuran graf menggunakan algoritma Depth First Search (DFS) dan Breadth First Search (BFS).
- Program bekerja dengan merepresentasikan graf menggunakan struktur data adjacency list yang disimpan dalam HashMap.
- Method `addEdge_3027()` digunakan untuk menambahkan hubungan antar simpul pada graf tak berarah.
- Method `printGraph_3027()` digunakan untuk menampilkan struktur graf dalam bentuk adjacency list.
- Method `dfs_3027()` digunakan untuk melakukan penelusuran graf secara Depth First Search dengan bantuan method rekursif `dfsHelper_3027()`.
- Method `bfs_3027()` digunakan untuk melakukan penelusuran graf secara Breadth First Search dengan memanfaatkan struktur data Queue.
- Pada method `main()`, program membuat graf sederhana, menampilkan graf awal, kemudian menjalankan algoritma DFS dan BFS.
- Hasil akhir program berupa urutan kunjungan simpul yang dihasilkan oleh algoritma DFS dan BFS.

```
package pekan9_2511533027;

import java.util.*;

public class GraphTraversal_2511533027 {
    private Map<String, List<String>> graph_3027 = new HashMap<>();

    // Menambahkan edge (graf tak berarah)
    public void addEdge_3027(String node1_3027, String node2_3027) {
        graph_3027.putIfAbsent(node1_3027, new ArrayList<>());
        graph_3027.putIfAbsent(node2_3027, new ArrayList<>());
        graph_3027.get(node1_3027).add(node2_3027);
        graph_3027.get(node2_3027).add(node1_3027);
    }

    // Menampilkan graf awal
    public void printGraph_3027() {
        System.out.println("Graf Awal (Adjacency List):");
        for (String node_3027 : graph_3027.keySet()) {
```

```

        System.out.print(node_3027 + " -> ");
        List<String> neighbors_3027 = graph_3027.get(node_3027);
        System.out.println(String.join(", ", neighbors_3027));
    }
    System.out.println();
}

// DFS rekursif
public void dfs_3027(String start_3027) {
    Set<String> visited_3027 = new HashSet<>();
    System.out.println("Penelusuran DFS:");
    dfsHelper_3027(start_3027, visited_3027);
    System.out.println();
}

private void dfsHelper_3027(String current_3027, Set<String>
visited_3027) {
    if (visited_3027.contains(current_3027)) return;
    visited_3027.add(current_3027);
    System.out.print(current_3027 + " ");
    for (String neighbor_3027 :
graph_3027.getDefault(current_3027, new ArrayList<>())) {
        dfsHelper_3027(neighbor_3027, visited_3027);
    }
}

// BFS iteratif
public void bfs_3027(String start_3027) {
    Set<String> visited_3027 = new HashSet<>();
    Queue<String> queue_3027 = new LinkedList<>();

    queue_3027.add(start_3027);
    visited_3027.add(start_3027);

    System.out.println("Penelusuran BFS:");
    while (!queue_3027.isEmpty()) {
        String current_3027 = queue_3027.poll();
        System.out.print(current_3027 + " ");
    }
}

```

```

        for (String neighbor :
graph_3027.getDefault(current_3027, new ArrayList<>())) {
            if (!visited_3027.contains(neighbor)) {
                queue_3027.add(neighbor);
                visited_3027.add(neighbor);
            }
        }
    }
    System.out.println();
}

// Main
public static void main(String[] args) {
    GraphTraversal_2511533027 graph_3027 = new
GraphTraversal_2511533027();

    // Contoh graf: A-B, A-C, B-D, B-E
    graph_3027.addEdge_3027("A", "B");
    graph_3027.addEdge_3027("A", "C");
    graph_3027.addEdge_3027("B", "D");
    graph_3027.addEdge_3027("B", "E");

    // Cetak graf awal
    System.out.println("Graf Awal adalah: ");
    graph_3027.printGraph_3027();

    // Lakukan penelusuran
    graph_3027.dfs_3027("A");
    graph_3027.bfs_3027("A");
}
}

```

4. Node_2511533027.java

- Program ini digunakan untuk merepresentasikan simpul (node) pada struktur data Binary Tree.
- Setiap simpul menyimpan data berupa bilangan integer serta memiliki referensi ke anak kiri dan anak kanan.

- Method `setLeft_3027()` dan `setRight_3027()` digunakan untuk menghubungkan simpul dengan anak kiri maupun anak kanan.
- Method `getLeft_3027()`, `getRight_3027()`, dan `getData_3027()` digunakan untuk mengakses informasi yang tersimpan pada simpul.
- Method `printInorder_3027()`, `printPreorder_3027()`, dan `printPostorder_3027()` digunakan untuk melakukan traversal pohon secara rekursif.
- Method `print_3027()` digunakan untuk menampilkan struktur Binary Tree dalam bentuk visual menyerupai pohon.
- Program ini berperan sebagai komponen utama yang digunakan oleh kelas `BTree` dalam membangun dan mengelola struktur Binary Tree.

```
package pekan9_2511533027;

import org.w3c.dom.Node;

public class Node_2511533027 {
    int data_3027;
    Node_2511533027 left_3027;
    Node_2511533027 right_3027;
    public Node_2511533027(int data_3027) {
        this.data_3027 = data_3027;
        left_3027 = null;
        right_3027 = null;
    }

    public void setLeft_3027(Node_2511533027 node_3027) {
        if (left_3027 == null)
            left_3027 = node_3027;
    }

    public void setRight_3027(Node_2511533027 node_3027) {
        if (right_3027 == null)
            right_3027 = node_3027;
    }

    public Node_2511533027 getLeft_3027() {
        return left_3027;
    }
}
```

```

}

public Node_2511533027 getRight_3027() {
    return right_3027;
}

public int getData_3027() {
    return data_3027;
}

public void setData_3027(int data_3027) {
    this.data_3027 = data_3027;
}

void printPreorder_3027(Node_2511533027 node_3027) {
    if (node_3027 == null)
        return;
    System.out.print(node_3027.data_3027 + " ");
    printPreorder_3027(node_3027.left_3027);
    printPreorder_3027(node_3027.right_3027);
}

void printPostorder_3027(Node_2511533027 node_3027) {
    if (node_3027 == null)
        return;
    printPostorder_3027(node_3027.left_3027);
    printPostorder_3027(node_3027.right_3027);
    System.out.print(node_3027.data_3027 + " ");
}

void printInorder_3027(Node_2511533027 node_3027) {
    if (node_3027 == null)
        return;
    printInorder_3027(node_3027.left_3027);
    System.out.print (node_3027.data_3027 + " ");
    printInorder_3027(node_3027.right_3027);
}

public String print_3027() {
    return this.print_3027("",true,"");
}

```



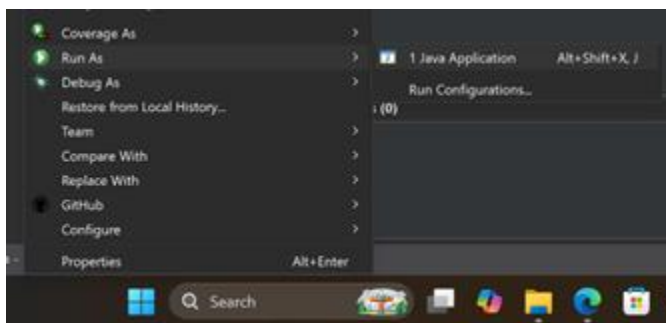
```

    }

    public String print_3027(String prefix_3027, boolean isTail_3027,
String sb_3027) {
        if (right_3027 != null) {
            right_3027.print_3027(prefix_3027 + (isTail_3027 ? "|"
: "  "), false, sb_3027);
        }
        System.out.println ( prefix_3027+(isTail_3027 ? "\\-- "
: "/-- ") +data_3027);
        if (left_3027 != null) {
            left_3027.print_3027(prefix_3027+ (isTail_3027 ?"  " :
"|  "), true, sb_3027);
        }
        return sb_3027;
    }
}

```

5. Kompilasi dan Menjalankan Program



2.2 Analisis Hasil dan Pembahasan

1. BTree_2511533027.java

- Hasil: Program berhasil mengimplementasikan operasi dasar pada struktur data Binary Tree. Program dapat melakukan pencarian data, menghitung jumlah simpul, serta menampilkan isi pohon menggunakan metode traversal Inorder, Preorder, dan Postorder. Selain itu, program juga mampu menampilkan struktur pohon dalam bentuk visual sehingga hubungan antara root, anak kiri, dan anak kanan dapat terlihat dengan jelas.
- Pembahasan: Pada program ini, kelas BTree berfungsi sebagai pengelola utama struktur Binary Tree. Method search_3027() digunakan untuk mencari data pada setiap simpul secara rekursif hingga data ditemukan atau seluruh simpul telah

diperiksa. Method `countNodes_3027()` digunakan untuk menghitung jumlah simpul yang terdapat dalam pohon dengan mengunjungi setiap simpul satu per satu. Program juga menyediakan method traversal yaitu Inorder, Preorder, dan Postorder yang menghasilkan urutan kunjungan simpul yang berbeda sesuai aturan masing-masing metode. Dari hasil pengujian terlihat bahwa seluruh operasi pada Binary Tree dapat dijalankan dengan baik dan menghasilkan keluaran yang sesuai dengan struktur pohon yang dibentuk.

2. BtreeDriver_2511533027.java

- a) Hasil: Program berhasil membentuk struktur Binary Tree yang terdiri dari sembilan simpul. Program mampu menampilkan jumlah simpul sebelum dan sesudah penambahan data, menampilkan nilai root, serta menghasilkan urutan traversal Inorder, Preorder, dan Postorder. Selain itu, program juga berhasil menampilkan bentuk pohon secara visual sehingga susunan antar simpul dapat diamati dengan lebih mudah.
- b) Pembahasan: Pada program ini dilakukan proses pembuatan simpul menggunakan kelas Node kemudian setiap simpul dihubungkan melalui anak kiri dan anak kanan sehingga membentuk Binary Tree. Simpul bernilai 1 ditetapkan sebagai root, kemudian ditambahkan beberapa simpul lainnya sesuai struktur yang diinginkan. Setelah pohon terbentuk, program melakukan pengujian dengan menghitung jumlah simpul dan menampilkan hasil traversal. Hasil traversal menunjukkan bahwa setiap metode memiliki urutan kunjungan yang berbeda. Inorder mengunjungi subtree kiri, root, lalu subtree kanan. Preorder mengunjungi root terlebih dahulu, sedangkan Postorder mengunjungi root terakhir. Hasil yang diperoleh menunjukkan bahwa struktur pohon telah terbentuk dengan benar.

3. GraphTraversal_2511533027.java

- a) Hasil: Program berhasil membentuk graf sederhana menggunakan representasi adjacency list. Program mampu menampilkan hubungan antar simpul dalam graf serta melakukan penelusuran menggunakan algoritma Depth First Search (DFS) dan Breadth First Search (BFS). Hasil penelusuran ditampilkan dalam bentuk urutan simpul yang dikunjungi mulai dari simpul awal yang dipilih.
- b) Pembahasan: Pada program ini, graf direpresentasikan menggunakan HashMap yang menyimpan daftar tetangga dari setiap simpul. Method `addEdge_3027()` digunakan untuk menambahkan hubungan antar simpul sehingga terbentuk graf tak berarah. Algoritma DFS dijalankan menggunakan pendekatan rekursif yang

menelusuri simpul hingga mencapai simpul terdalam sebelum kembali ke simpul sebelumnya. Sementara itu, algoritma BFS menggunakan struktur data Queue untuk menelusuri simpul berdasarkan tingkat kedekatannya dari simpul awal. Dari hasil percobaan terlihat bahwa urutan kunjungan simpul pada DFS dan BFS berbeda. DFS cenderung menelusuri satu jalur hingga selesai terlebih dahulu, sedangkan BFS mengunjungi seluruh simpul yang berada pada tingkat yang sama sebelum berpindah ke tingkat berikutnya.

4. Node_2511533027.java

- a) Hasil: Program berhasil merepresentasikan simpul pada struktur Binary Tree. Setiap simpul dapat menyimpan data, memiliki hubungan dengan anak kiri dan anak kanan, serta mendukung proses traversal pohon. Program juga mampu menampilkan struktur pohon dalam bentuk visual yang memudahkan pengguna dalam memahami susunan simpul pada Binary Tree.
- b) Pembahasan: Pada program ini, kelas Node berfungsi sebagai komponen dasar pembentuk Binary Tree. Setiap objek Node menyimpan data berupa bilangan integer dan referensi ke anak kiri maupun anak kanan. Method setter dan getter digunakan untuk mengatur serta mengambil informasi dari simpul. Selain itu, kelas ini menyediakan method traversal Inorder, Preorder, dan Postorder yang bekerja secara rekursif untuk mengunjungi setiap simpul sesuai aturan masing-masing metode. Method print_3027() digunakan untuk menampilkan struktur pohon dalam bentuk visual sehingga hubungan antar simpul dapat terlihat dengan lebih jelas. Dari hasil pengujian dapat disimpulkan bahwa kelas Node telah berfungsi dengan baik sebagai dasar pembentukan dan pengelolaan Binary Tree.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, struktur data Binary Tree dan Graph berhasil diimplementasikan menggunakan bahasa pemrograman Java. Pada Binary Tree, program mampu membentuk hubungan antar simpul, menghitung jumlah simpul, serta menampilkan hasil traversal menggunakan metode Inorder, Preorder, dan Postorder. Selain itu, struktur pohon juga dapat ditampilkan dalam bentuk visual sehingga memudahkan pemahaman mengenai hubungan antara root, anak kiri, dan anak kanan.

Implementasi algoritma Depth First Search (DFS) dan Breadth First Search (BFS) pada graph juga berjalan dengan baik. Hasil penelusuran menunjukkan bahwa DFS menelusuri simpul hingga ke tingkat terdalam terlebih dahulu, sedangkan BFS menelusuri simpul berdasarkan tingkat kedekatannya dari simpul awal. Melalui praktikum ini, pemahaman mengenai konsep tree, graph, serta algoritma BFS dan DFS menjadi lebih baik, sekaligus meningkatkan kemampuan dalam mengimplementasikan struktur data dan algoritma penelusuran pada pemrograman Java.

DAFTAR PUSTAKA

- [1] Wahyudi, *Materi Struktur Data: Sorting*. Padang: PPT Perkuliahan, 2026.
- [2] M. A. Weiss, *Data Structures and Algorithm Analysis in Java*, 3rd ed. Boston, MA, USA: Pearson Education, 2012.

LAPORAN TUGAS PRAKTIKUM

STRUKTUR DATA

“BFS DAN DFS”



disusun Oleh:

NAIRA RAMADHANI HALIL

2511533027

Dosen Pengampu:

Dr. WAHYUDI, S.T, M.T.

Asisten Praktikum:

ZAHRAN AZARIA ANVAYA

DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Dengan memanjatkan puji syukur kehadiran Allah SWT, atas rahmat dan karunia-Nya sehingga laporan tugas praktikum Struktur Data ini dapat diselesaikan dengan baik. Shalawat serta salam semoga selalu tercurah kepada Nabi Muhammad SAW.

Laporan tugas ini disusun untuk memenuhi tugas praktikum Struktur Data dengan topik Sorting menggunakan studi kasus “Peta Map” dengan nama class TransmartPadang_2511533027. Pada praktikum ini, penulis membuat program Java yang dapat mengelola peta Transmart Padang menggunakan metode BFS dan DFS.

Penulis mengucapkan terima kasih kepada dosen pengampu dan asisten praktikum atas bimbingannya. Penulis juga menyadari bahwa laporan ini masih memiliki kekurangan, sehingga kritik dan saran sangat diharapkan.

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI.....	ii
BAB I PENDAHULUAN.....	1
1.1 Latar Belakang	1
1.2 Tujuan	1
1.3 Manfaat	2
1.4 Instruksi Tugas	2
BAB II PEMBAHASAN.....	4
2.1 Kelas TransmartPadang_2511533027.....	4
BAB III PENUTUP	13
3.1 Kesimpulan	13
DAFTAR PUSTAKA	14

BAB I

PENDAHULUAN

1.1 Latar Belakang

Graf merupakan salah satu struktur data yang digunakan untuk merepresentasikan hubungan antar objek dalam bentuk simpul (vertex) dan sisi (edge). Struktur data ini banyak diterapkan dalam kehidupan sehari-hari, seperti pencarian rute perjalanan, jaringan komputer, media sosial, hingga sistem navigasi. Untuk melakukan pencarian pada graf, terdapat beberapa algoritma yang umum digunakan, di antaranya Breadth First Search (BFS) dan Depth First Search (DFS).

Breadth First Search (BFS) merupakan algoritma penelusuran yang bekerja dengan mengunjungi simpul berdasarkan tingkat kedekatannya dari simpul awal. Sementara itu, Depth First Search (DFS) melakukan penelusuran dengan menelusuri satu jalur sedalam mungkin sebelum kembali ke simpul sebelumnya. Kedua algoritma tersebut memiliki karakteristik dan cara kerja yang berbeda sehingga penting untuk dipahami melalui implementasi secara langsung.

Pada praktikum ini dilakukan implementasi algoritma BFS dan DFS menggunakan bahasa pemrograman Java berbasis GUI. Studi kasus yang digunakan adalah pemetaan beberapa lokasi di Transmart Padang dalam bentuk graf. Melalui visualisasi graf dan proses penelusuran yang ditampilkan secara interaktif, mahasiswa dapat memahami cara kerja BFS dan DFS serta membandingkan hasil pencarian jalur yang diperoleh dari kedua algoritma tersebut.

1.2 Tujuan

1. Memahami konsep struktur data graf beserta komponennya yang terdiri dari vertex dan edge.
2. Memahami prinsip kerja algoritma Breadth First Search (BFS) dan Depth First Search (DFS).
3. Mengimplementasikan algoritma BFS dan DFS menggunakan bahasa pemrograman Java.
4. Membuat visualisasi graf menggunakan Java Swing GUI.
5. Menentukan jalur dari lokasi awal menuju lokasi tujuan berdasarkan algoritma BFS dan DFS.

6. Menganalisis hasil penelusuran yang dihasilkan oleh kedua algoritma pada suatu graf.

1.3 Manfaat

1. Menambah pemahaman mengenai penerapan struktur data graf dalam pemrograman.
2. Melatih kemampuan dalam mengimplementasikan algoritma BFS dan DFS menggunakan Java.
3. Membantu memahami perbedaan karakteristik BFS dan DFS dalam proses pencarian jalur.
4. Meningkatkan kemampuan dalam merancang antarmuka grafis menggunakan Java Swing.
5. Memberikan gambaran penerapan algoritma graf pada permasalahan nyata, seperti pencarian lokasi dan navigasi.

1.4 Instruksi Tugas

1. Penamaan Class dan File
Gunakan akhiran NIM lengkap pada nama class dan nama file.
Contoh:
ex : PetaKampus_2311532008
2. Penamaan Variabel
Gunakan 4 digit terakhir pada setiap nama variabel
ex : graph_2019
3. Struktur Tugas
 - a) Vertex and Edge
 - Minimal 10 simpul (vertex)
 - Minimal 15 sisi (edge)
 - Nama simpul harus merepresentasikan lokasi pada peta yang dibuat
 - b) Method yang Wajib Ada
 - BFS() → Melakukan pencarian menggunakan algoritma Breadth First Search.
 - DFS() → Melakukan pencarian menggunakan algoritma Depth First Search.
 - displayGraph() → Menampilkan graph pada GUI.
 - displayPath() → Menampilkan jalur yang ditemukan.
 - resetGraph() → Mengembalikan kondisi graph ke keadaan awal.
 - c) GUI Program
GUI minimal memiliki:
 - Area visualisasi graph

- ComboBox atau TextField untuk memilih lokasi awal
- ComboBox atau TextField untuk memilih lokasi tujuan
- Tombol BFS
- Tombol DFS
- Tombol Reset
- Area hasil pencarian
- Berikan warna berbeda pada node yang telah dikunjungi
- Menampilkan jumlah node yang dieksplorasi

d) Ketentuan Graph

- Setiap mahasiswa wajib membuat graph yang berbeda.
- Tidak diperbolehkan menggunakan graph yang sama persis dengan mahasiswa lain.
- Graph harus memiliki minimal 10 node dan 15 edge.
- Mahasiswa bebas menentukan bentuk dan jenis peta.
- Mahasiswa wajib menjelaskan struktur graph, cara kerja BFS dan DFS yang dibuat pada laporan.

4. Output yang Dikumpulkan

- a) Source code Java.
- b) Screenshot program saat dijalankan.
- c) Laporan singkat PDF) yang berisi:
 - Deskripsi peta yang dibuat.
 - Jumlah node dan edge.
 - Hasil BFS.
 - Hasil DFS.
 - Kesimpulan perbandingan BFS dan DFS.

BAB II PEMBAHASAN

2.1 Kelas TransmartPadang_2511533027

Dalam tugas ini, saya mengembangkan program TransmartPadang_2511533027.java untuk mengimplementasikan algoritma BFS dan DFS pada graf yang merepresentasikan area Transmart Padang. Program ini dirancang menggunakan Java Swing GUI agar lebih mudah dimengerti, di mana pengguna cukup memilih titik awal dan tujuan melalui menu ComboBox. Struktur grafnya sendiri dibentuk dari beberapa lokasi utama, seperti Pintu Utama, Customer Service, Hypermarket, Food Court, ATM Center, dan Area Parkir sebagai simpul, serta jalur penghubungnya sebagai sisi.

Penerapan kedua algoritma ini punya karakteristik yang sengaja dibedakan: BFS akan menjelajahi simpul tetangga terdekat terlebih dahulu, sementara DFS akan menyelami satu jalur hingga titik terdalam sebelum *backtracking*. Output yang dihasilkan program meliputi urutan kunjungan, jalur terbaik, dan jumlah simpul yang diperiksa. Untuk mempermudah pemahaman, program ini dilengkapi visualisasi interaktif berupa perubahan warna pada simpul selama proses pencarian berlangsung, sehingga perbedaan logis antara BFS dan DFS bisa terlihat jelas.

```
package pekan9_2511533027;

import java.awt.BasicStroke;
import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.EventQueue;
import java.awt.Graphics;
import java.awt.Graphics2D;
import java.awt.Point;
import java.util.ArrayList;
import java.util.Collections;
import java.util.HashMap;
import java.util.LinkedHashMap;
import java.util.LinkedHashSet;
import java.util.LinkedList;
import java.util.List;
import java.util.Map;
import java.util.Queue;
import java.util.Set;
import java.util.Stack;
import java.util.stream.Collectors;

import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
```

```

import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import javax.swing.SwingUtilities;
import javax.swing.Timer;
import javax.swing.border.EmptyBorder;

public class TransmartPadang_2511533027 extends JFrame {

    private JComboBox<String> startCombo_3027;
    private JComboBox<String> goalCombo_3027;
    private JButton bfsButton_3027;
    private JButton dfsButton_3027;
    private JButton resetButton_3027;
    private JTextArea resultArea_3027;
    private GraphPanel_3027 graphPanel_3027;
    private Graph_3027 graph_3027;

    public TransmartPadang_2511533027() {
        graph_3027 = new Graph_3027();

        setTitle("BFS & DFS - Transmart Padang (2511533027)");
        setSize(1100, 800);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(EXIT_ON_CLOSE);

        initializeGUI_3027();

        setVisible(true);
    }
    private void initializeGUI_3027() {
        JPanel topPanel_3027 = new JPanel();

        startCombo_3027 = new
JComboBox<>(graph_3027.getVertices_3027().toArray(new String[0]));

        goalCombo_3027 = new
JComboBox<>(graph_3027.getVertices_3027().toArray(new String[0]));

        bfsButton_3027 = new JButton("BFS");
        dfsButton_3027 = new JButton("DFS");
        resetButton_3027 = new JButton("RESET");

        topPanel_3027.add(new JLabel("Start"));
        topPanel_3027.add(startCombo_3027);

        topPanel_3027.add(new JLabel("Goal"));
        topPanel_3027.add(goalCombo_3027);

        topPanel_3027.add(bfsButton_3027);
        topPanel_3027.add(dfsButton_3027);
        topPanel_3027.add(resetButton_3027);

        add(topPanel_3027, BorderLayout.NORTH);

        graphPanel_3027 = new GraphPanel_3027();

        add(graphPanel_3027, BorderLayout.CENTER);
    }
}

```

```

        resultArea_3027 = new JTextArea(4, 20);
        resultArea_3027.setEditable(false);

        JScrollPane scrollPane_3027 = new JScrollPane(resultArea_3027);
        add(scrollPane_3027, BorderLayout.SOUTH);

        bfsButton_3027.addActionListener(
            e -> runBFS_3027());

        dfsButton_3027.addActionListener(
            e -> runDFS_3027());

        resetButton_3027.addActionListener(
            e -> resetGraph_3027());
    }

    private void runBFS_3027() {
        String start_3027 = (String) startCombo_3027.getSelectedItem();
        String goal_3027 = (String) goalCombo_3027.getSelectedItem();
        SearchResult_3027 result_3027 = graph_3027.bfs_3027(start_3027,
goal_3027);

        animateTraversal_3027("BFS", result_3027);
    }

    private void runDFS_3027() {
        String start_3027 = (String) startCombo_3027.getSelectedItem();
        String goal_3027 = (String) goalCombo_3027.getSelectedItem();
        SearchResult_3027 result_3027 = graph_3027.dfs_3027(start_3027,
goal_3027);

        animateTraversal_3027("DFS", result_3027);
    }

    private void resetGraph_3027() {
        graphPanel_3027.resetGraph_3027();

        resultArea_3027.setText("");
    }

    private void animateTraversal_3027(String algorithm_3027, SearchResult_3027
result_3027) {
        graphPanel_3027.resetGraph_3027();

        List<String> visited_3027 = result_3027.getVisited_3027();

        Timer timer_3027 = new Timer(700, null);

        final int[] index_3027 = {0};

```

```

        timer_3027.addActionListener(
            e -> {
                if (index_3027[0] < visited_3027.size()) {
graphPanel_3027.visitNode_3027(visited_3027.get(index_3027[0]));
                    index_3027[0]++;
                } else {
                    timer_3027.stop();

graphPanel_3027.setFinalPath_3027(result_3027.getPath_3027());
                    displayResult_3027(
                        algorithm_3027,
                        result_3027);
                }
            });

        timer_3027.start();
    }

    private void displayResult_3027(String algorithm_3027, SearchResult_3027
result_3027) {
        String output_3027 = "Algoritma : " + algorithm_3027 + "\n\n" +
"Urutan Kunjungan :\n" +
result_3027.getVisited_3027().stream().collect(Collectors.joining(" -> "))
            + "\n\nPath :\n" +
result_3027.getPath_3027().stream().collect(Collectors.joining(" -> "))
            + "\n\nJumlah Node Dieksplorasi : " +
result_3027.getExploredNodes_3027();
        resultArea_3027.setText(output_3027);
    }

    class SearchResult_3027 {
        private List<String> path_3027;
        private List<String> visited_3027;
        private int exploredNodes_3027;
        public SearchResult_3027(List<String> path_3027, List<String>
visited_3027, int exploredNodes_3027) {
            this.path_3027 = path_3027;
            this.visited_3027 = visited_3027;
            this.exploredNodes_3027 = exploredNodes_3027;
        }

        public List<String> getPath_3027() {
            return path_3027;
        }

        public List<String> getVisited_3027() {
            return visited_3027;
        }

        public int getExploredNodes_3027() {
            return exploredNodes_3027;
        }
    }

    class Graph_3027 {
        private Map<String, List<String>> graph_3027;
        public Graph_3027() {
            graph_3027 = new LinkedHashMap<>();

```

```

        buildGraph_3027();
    }

    private void addEdge_3027(String source_3027, String destination_3027)
    {
        graph_3027.putIfAbsent(source_3027, new ArrayList<>());
        graph_3027.putIfAbsent(destination_3027, new ArrayList<>());
        graph_3027.get(source_3027).add(destination_3027);
        graph_3027.get(destination_3027).add(source_3027);
    }

    private void buildGraph_3027() {
        addEdge_3027("Pintu Utama", "Customer Service");
        addEdge_3027("Pintu Utama", "ATM Center");
        addEdge_3027("Pintu Utama", "Area Parkir");
        addEdge_3027("Customer Service", "Hypermarket");
        addEdge_3027("Customer Service", "Food Court");
        addEdge_3027("Hypermarket", "Food Court");
        addEdge_3027("Hypermarket", "Mushalla");
        addEdge_3027("Hypermarket", "Eskalator");
        addEdge_3027("Food Court", "Bioskop XXI");
        addEdge_3027("Food Court", "Timezone");
        addEdge_3027("ATM Center", "Mushalla");
        addEdge_3027("ATM Center", "Area Parkir");
        addEdge_3027("Eskalator", "Bioskop XXI");
        addEdge_3027("Eskalator", "Timezone");
        addEdge_3027("Bioskop XXI", "Timezone");
    }

    public Set<String> getVertices_3027() {
        return graph_3027.keySet();
    }

    public Map<String, List<String>> getGraph_3027() {
        return graph_3027;
    }

    public SearchResult_3027 bfs_3027(String start_3027, String goal_3027)
    {

```



```

Queue<String> queue_3027 = new LinkedList<>();

Set<String> visited_3027 = new LinkedHashSet<>();

Map<String, String> parent_3027 = new HashMap<>();

queue_3027.add(start_3027);

visited_3027.add(start_3027);

while (!queue_3027.isEmpty()) {
    String current_3027 =
        queue_3027.poll();
    if (current_3027.equals(
        goal_3027)) {
        break;
    }
    for (String neighbor_3027 : graph_3027.get(current_3027)) {
        if (!visited_3027.contains(neighbor_3027)) {
            visited_3027.add(neighbor_3027);

            parent_3027.put(neighbor_3027, current_3027);

            queue_3027.add(neighbor_3027);
        }
    }
}
return buildResult_3027(goal_3027, parent_3027, visited_3027);
}

public SearchResult_3027 dfs_3027(String start_3027, String goal_3027) {
    Stack<String> stack_3027 = new Stack<>();

    Set<String> visited_3027 = new LinkedHashSet<>();

    Map<String, String> parent_3027 = new HashMap<>();

    stack_3027.push(start_3027);

    while (!stack_3027.isEmpty()) {
        String current_3027 = stack_3027.pop();
        if (!visited_3027.contains(current_3027)) {
            visited_3027.add(current_3027);
            if (current_3027.equals(goal_3027)) {
                break;
            }
        }
        List<String> neighbors_3027 =
graph_3027.get(current_3027);
        for (int i_3027 = neighbors_3027.size() - 1;
            i_3027 >= 0;
            i_3027--) {
            String next_3027 = neighbors_3027.get(i_3027);
            if (!visited_3027.contains(next_3027)) {
                parent_3027.put(next_3027, current_3027);
                stack_3027.push(next_3027);
            }
        }
    }
}

```

```

    }
    return buildResult_3027(
        goal_3027,
        parent_3027,
        visited_3027);
}

private SearchResult_3027 buildResult_3027(String goal_3027,
Map<String, String> parent_3027, Set<String> visited_3027) {
    List<String> path_3027 = new ArrayList<>();

    String current_3027 = goal_3027;
    while (current_3027 != null) {
        path_3027.add(current_3027);
        current_3027 = parent_3027.get(current_3027);
    }
    Collections.reverse(path_3027);
    return new SearchResult_3027(path_3027, new
ArrayList<>(visited_3027), visited_3027.size());
}

}

class GraphPanel_3027 extends JPanel {
    private Map<String, Point> positions_3027;
    private Set<String> visitedNodes_3027;
    private String currentNode_3027;
    private List<String> finalPath_3027;
    public GraphPanel_3027() {
        visitedNodes_3027 = new LinkedHashSet<>();

        finalPath_3027 = new ArrayList<>();
        initializePositions_3027();
        setBackground(Color.WHITE);
    }
    private void initializePositions_3027() {
        positions_3027 = new HashMap<>();
        positions_3027.put("Bioskop XXI", new Point(450, 80));
        positions_3027.put("Timezone", new Point(650, 80));
        positions_3027.put("Eskalator", new Point(550, 180));
        positions_3027.put("Food Court", new Point(550, 300));
        positions_3027.put("Hypermarket", new Point(350, 300));
        positions_3027.put("Customer Service", new Point(750, 300));
        positions_3027.put("Mushalla", new Point(350, 450));
        positions_3027.put("ATM Center", new Point(750, 450));
        positions_3027.put("Pintu Utama", new Point(450, 600));
        positions_3027.put("Area Parkir", new Point(650, 600));
    }

    public void visitNode_3027(String node_3027) {
        currentNode_3027 = node_3027;
        visitedNodes_3027.add(node_3027);
        repaint();
    }

    public void setFinalPath_3027(List<String> path_3027) {
        finalPath_3027 = path_3027;
        currentNode_3027 = null;
        repaint();
    }
}

```

```

public void resetGraph_3027() {
    visitedNodes_3027.clear();
    finalPath_3027.clear();
    currentNode_3027 = null;
    repaint();
}

@Override
protected void paintComponent(Graphics g_3027) {
    super.paintComponent(g_3027);
    Graphics2D g2_3027 = (Graphics2D) g_3027;
    drawEdges_3027(g2_3027);
    drawNodes_3027(g2_3027);
}

private void drawEdges_3027(Graphics2D g2_3027) {
    for (String node_3027 : graph_3027.getGraph_3027().keySet()) {
        Point p1_3027 = positions_3027.get(node_3027);
        for (String neighbor_3027 :
graph_3027.getGraph_3027().get(node_3027)) {
            Point p2_3027 = positions_3027.get(neighbor_3027);
            boolean isPath_3027 = isEdgeInPath_3027(node_3027,
neighbor_3027);

            if (isPath_3027) {
                g2_3027.setColor(Color.RED);
                g2_3027.setStroke(new BasicStroke(4));
            } else {
                g2_3027.setColor(Color.GRAY);
                g2_3027.setStroke(new BasicStroke(2));
            }
            g2_3027.drawLine(p1_3027.x, p1_3027.y, p2_3027.x,
p2_3027.y);
        }
    }
}

private boolean isEdgeInPath_3027(String source_3027, String destination_3027)
{
    for (int i_3027 = 0; i_3027 < finalPath_3027.size() - 1; i_3027++)
    {
        String a_3027 = finalPath_3027.get(i_3027);
        String b_3027 = finalPath_3027.get(i_3027 + 1);
        if ((a_3027.equals(source_3027) &&
b_3027.equals(destination_3027)) ||
            (a_3027.equals(destination_3027) &&
b_3027.equals(source_3027))) {
            return true;
        }
    }
    return false;
}

private void drawNodes_3027(Graphics2D g2_3027) {
    for (String node_3027 : positions_3027.keySet()) {
        Point p_3027 = positions_3027.get(node_3027);
        if (node_3027.equals(currentNode_3027)) {
            g2_3027.setColor(Color.YELLOW);
        }
    }
}

```

```

        else if (finalPath_3027.contains(node_3027)) {
            g2_3027.setColor(Color.RED);
        }
        else if (visitedNodes_3027.contains(node_3027)) {
            g2_3027.setColor(Color.GREEN);
        } else {
            g2_3027.setColor(Color.CYAN);
        }
        g2_3027.fillOval(p_3027.x - 25, p_3027.y - 25, 50, 50);
        g2_3027.setColor(Color.BLACK);
        g2_3027.drawOval(p_3027.x - 25, p_3027.y - 25, 50, 50);
        g2_3027.drawString(node_3027, p_3027.x - 40, p_3027.y - 35);
    }
}

public static void main(
    String[] args) {

    SwingUtilities.invokeLater(
        () -> new TransmartPadang_2511533027());
    }
}

```

BAB III PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum yang telah dilakukan, algoritma Breadth First Search (BFS) dan Depth First Search (DFS) berhasil diimplementasikan menggunakan bahasa pemrograman Java dengan antarmuka berbasis GUI. Graf yang digunakan untuk merepresentasikan lokasi-lokasi di Transmart Padang dapat ditelusuri dengan baik menggunakan kedua algoritma tersebut. Program juga mampu menampilkan urutan kunjungan simpul, jalur yang ditemukan, serta jumlah simpul yang dieksplorasi selama proses pencarian.

Melalui visualisasi yang ditampilkan, dapat dipahami bahwa BFS dan DFS memiliki cara kerja yang berbeda dalam menelusuri graf. BFS cenderung menelusuri simpul berdasarkan tingkat kedekatan dari titik awal, sedangkan DFS menelusuri satu jalur hingga sedalam mungkin sebelum berpindah ke jalur lain. Dengan adanya implementasi dan visualisasi ini, pemahaman mengenai konsep graf, traversal graf, serta penerapan algoritma BFS dan DFS menjadi lebih mudah dipahami dan diamati secara langsung.

DAFTAR PUSTAKA

- [1] Wahyudi, *Materi Struktur Data: Sorting*. Padang: PPT Perkuliahan, 2026.
- [2] M. A. Weiss, *Data Structures and Algorithm Analysis in Java*, 3rd ed. Boston, MA, USA: Pearson Education, 2012.